Arquitectura y procedimiento de despliegue del Sistema de Suscripciones

Sergio Mesa

Sistema de Suscripciones

Documento técnico para revisión

16 de octubre de 2026

Resumen

Este documento presenta la arquitectura de software, las decisiones técnicas y el procedimiento de despliegue del Sistema de Suscripciones. La solución fue construida con .NET 10, ASP.NET Core, MongoDB Atlas y Angular 22, siguiendo una separación por capas inspirada en Clean Architecture. Se describen las responsabilidades de Domain, Application, Infrastructure, Api y Web; la inversión de dependencias mediante interfaces; el ciclo de estados de los recordatorios; la persistencia documental; el tratamiento local de secretos; y la estrategia de pruebas. Asimismo, se ofrecen instrucciones reproducibles para la revisión local en modalidades separada e integrada y se establece una ruta de referencia para un despliegue productivo en Render mediante contenedores. El análisis identifica las limitaciones actuales de la configuración SMTP en producción y determina los controles que deben completarse antes de publicar el sistema.

Palabras clave: *arquitectura limpia, ASP.NET Core, Angular, MongoDB Atlas, SMTP, despliegue.*

Contenido

1. Introducción

2. Alcance técnico de la solución

3. Arquitectura del sistema

4. Decisiones técnicas

5. Funcionamiento de los procesos principales

6. Configuración y seguridad

7. Despliegue local para revisión técnica

8. Ruta de despliegue productivo en Render

9. Pruebas y criterios de aceptación

10. Riesgos, limitaciones y acciones pendientes

11. Conclusiones

Referencias

1. Introducción

El Sistema de Suscripciones administra facturas asociadas con socios y automatiza el avance de los recordatorios de pago. Su alcance funcional comprende el registro, consulta, modificación y eliminación de facturas; el envío de avisos individuales y masivos; la configuración local del servicio SMTP; la personalización de mensajes; y la generación de un documento PDF de carácter administrativo.

El diseño prioriza la separación de responsabilidades. La lógica del negocio no conoce detalles de MongoDB, SMTP, HTTP ni Angular. Las tecnologías externas se conectan mediante contratos definidos en la capa de aplicación. Esta organización facilita el mantenimiento, permite sustituir adaptadores y hace posible probar las reglas sin depender de servicios externos.

El documento refleja el estado actual del repositorio SistemaSuscripciones. Por tanto, diferencia entre las capacidades implementadas y las actividades que todavía serían necesarias para una publicación productiva. Esta distinción es esencial para una revisión técnica verificable.

2. Alcance técnico de la solución

Componente	**Tecnología**	**Responsabilidad**
Backend	.NET 10 y ASP.NET Core	API REST, composición de dependencias y alojamiento web.
Persistencia	MongoDB Atlas y MongoDB.Driver 3.12.0	Almacenamiento de facturas con el socio embebido.
Frontend	Angular 22	Interfaz, formularios, navegación interna y consumo HTTP.

Componente	Tecnología	Responsabilidad
Correo	System.Net.Mail	Entrega de recordatorios mediante un servidor SMTP.
PDF	jsPDF 4.2.1	Generación de un resumen administrativo en el navegador.
Documentación	Swashbuckle 10.3.0	Exposición del contrato OpenAPI y Swagger en desarrollo.
Pruebas	xUnit 2.9.3 y Vitest 5	Validación unitaria del backend y del frontend.

La solución contiene cuatro proyectos .NET, un proyecto de pruebas y una aplicación Angular. El backend se agrupa en SistemaSuscripciones.slnx. El frontend conserva su propio ciclo de dependencias y compilación mediante npm.

3. Arquitectura del sistema

3.1. Vista por capas

SistemaSuscripciones.Domain

↑

SistemaSuscripciones.Application

↑

SistemaSuscripciones.Infrastructure

↑

SistemaSuscripciones.Api ← HTTP → SistemaSuscripciones.Web

Las flechas representan la dependencia del código. Domain permanece independiente; Application depende del dominio; Infrastructure implementa contratos de Application; Api

ensambla el sistema; y Web se comunica exclusivamente a través de HTTP. Esta dirección evita que las reglas centrales queden ligadas a una base de datos, un proveedor de correo o una interfaz específica.

3.2. Capa Domain

SistemaSuscripciones.Domain contiene las entidades Factura y Socio, además del enum EstadoRecordatorio. Factura representa una obligación de pago e incluye identificador, número visible, socio, valor, vencimiento, estado y fecha UTC del último correo. Socio conserva nombre, correo y plan de suscripción. EstadoRecordatorio limita el ciclo a PrimerRecordatorio, SegundoRecordatorio y Desactivado.

Esta capa no contiene referencias a MongoDB, ASP.NET Core, SMTP ni Angular. Su bajo acoplamiento preserva el lenguaje del negocio y reduce el costo de evolución tecnológica.

3.3. Capa Application

SistemaSuscripciones.Application define los contratos IFacturaRepository, IEmailService e IRecordatorioService. También contiene los objetos de transferencia FacturaResumenDto, GuardarFacturaDto y ResultadoRecordatorioDto, junto con RecordatorioService. Los contratos expresan qué necesita el caso de uso, mientras Infrastructure decide cómo satisfacerlo.

RecordatorioService concentra la regla de transición. EnviarAsync busca la factura, valida su existencia y estado, envía el correo, cambia el estado, registra la fecha UTC y persiste. ProcesarPendientesAsync filtra las facturas desactivadas y procesa únicamente las activas.

3.4. Capa Infrastructure

SistemaSuscripciones.Infrastructure contiene FacturaMongoRepository, SmtpEmailService, SmtpSettingsStore y DependencyInjection. FacturaMongoRepository implementa las operaciones asíncronas de consulta, inserción, reemplazo y eliminación mediante

MongoDB.Driver. El socio se guarda embebido en el documento de factura, lo que simplifica la lectura para la tabla principal.

SmtpEmailService selecciona la plantilla correspondiente al estado actual y reemplaza variables como nombre del socio, número de factura, valor y fecha de vencimiento. SmtpSettingsStore utiliza ASP.NET Core Data Protection para cifrar la configuración SMTP antes de almacenarla localmente.

3.5. Capa Api

SistemaSuscripciones.Api es el punto de entrada del servidor. Program.cs carga la configuración, registra controladores, serializa enums como texto, habilita Swagger, configura CORS, compone Infrastructure y Application, aplica el manejo general de errores y, en modalidad integrada, sirve los archivos compilados de Angular.

Método	**Ruta**	**Responsabilidad**
GET	/api/Facturas	Consultar la lista de facturas.
POST	/api/Facturas	Crear una factura.
PUT	/api/Facturas/{id}	Actualizar datos editables.
DELETE	/api/Facturas/{id}	Eliminar una factura.
POST	/api/Facturas/{id}/enviar-recordatorio	Enviar el siguiente aviso.
POST	/api/Facturas/procesar-pendientes	Procesar todas las facturas activas.

Método	Ruta	Responsabilidad
GET/ PUT	/api/configuracion/smtp	Consultar o actualizar SMTP en Development.
POST	/api/configuracion/smtp/probar	Comprobar la entrega SMTP.

3.6. Capa Web

SistemaSuscripciones.Web implementa la experiencia de usuario con Angular 22. FacturasService encapsula el CRUD y las operaciones de recordatorio; ConfiguracionSmtpService administra la configuración local de correo; y FacturaPdfService genera el PDF mediante una importación dinámica de jsPDF. Angular 22 requiere una versión compatible de Node.js; la matriz oficial incluye Node 22.22.3, que coincide con la versión recomendada por el proyecto (Angular, 2026).

4. Decisiones técnicas

Separación por capas. Se adoptó para aislar las reglas de negocio de las tecnologías externas y permitir que el dominio permanezca estable.

Inversión de dependencias. Application declara interfaces y DependencyInjection conecta implementaciones concretas. Esta decisión permite usar dobles de prueba sin MongoDB ni SMTP.

Modelo documental. El socio se embebe en la factura porque la pantalla principal necesita ambos datos en una misma lectura y el alcance no exige administrar socios independientemente.

Operaciones asíncronas. Las consultas, escrituras y envíos utilizan Task para no bloquear los hilos del servidor durante operaciones de entrada y salida.

Enums como texto. La API serializa los estados por nombre para que Swagger, Angular y los registros sean legibles y menos sensibles al orden numérico del enum.

Cambio de estado posterior al envío. La factura avanza solamente cuando SMTP acepta el mensaje; un fallo conserva el estado y evita registrar una entrega inexistente.

Configuración externa. MongoDB se configura con variables de entorno. .NET admite doble guion bajo para representar jerarquías y las variables posteriores prevalecen sobre appsettings.json (Microsoft, 2026).

Protección del secreto SMTP local. La contraseña se cifra y no regresa al navegador. Los endpoints de configuración se restringen a Development.

Ejecución dual e integrada. Durante el desarrollo se conservan servidores independientes; para demostración, Angular se compila en wwwroot y ASP.NET Core sirve toda la aplicación.

PDF en el cliente. El documento se genera sin carga adicional para la API. Su alcance es administrativo y no fiscal.

Respuestas de error seguras. La API devuelve mensajes generales y registra únicamente el tipo de excepción, evitando exponer contraseñas o cadenas de conexión.

5. Funcionamiento de los procesos principales

5.1. Inicio de la aplicación

1. Program.cs determina el entorno mediante DOTNET_ENVIRONMENT o ASPNETCORE_ENVIRONMENT.

2. En Development, DotNetEnv busca el archivo .env de la API y lo carga sin sobrescribir variables ya definidas por el sistema operativo.

3. El proveedor de variables de entorno se agrega a la configuración para otorgarle prioridad sobre appsettings.json.

4. AddInfrastructure enlaza MongoDbSettings y registra repositorio, correo, almacenamiento SMTP y Data Protection.

5. ASP.NET Core construye la aplicación, configura middleware y publica los controladores.

5.2. Operaciones CRUD

El controlador recibe y valida GuardarFacturaDto. En la creación genera identificadores sin guiones para la factura y el socio, construye la entidad y solicita su inserción. En la actualización recupera el documento existente para conservar UltimoCorreoEnviado y reemplaza únicamente los datos editables. La eliminación devuelve 204 cuando MongoDB confirma la eliminación y 404 cuando el identificador no existe.

5.3. Envío de recordatorios

1. Angular envía el identificador al endpoint de recordatorio.

2. FacturasController delega el caso de uso a IRecordatorioService.

3. RecordatorioService consulta la factura por IFacturaRepository.

4. Si la factura no existe o está desactivada, se devuelve un resultado controlado sin correo.

5. IEmailService genera y entrega la plantilla correspondiente al estado vigente.

6. Después del envío, el estado avanza y UltimoCorreoEnviado recibe DateTime.UtcNow.

7. El repositorio reemplaza el documento y la API devuelve el nuevo estado.

5.4. Procesamiento masivo

ProcesarPendientesAsync consulta todas las facturas y descarta las que se encuentran en Desactivado. Cada factura activa sigue el mismo flujo individual. La implementación actual es secuencial, decisión conservadora que reduce envíos simultáneos y simplifica la consistencia.

Para un volumen elevado sería conveniente incorporar una cola de trabajos, reintentos controlados e idempotencia.

5.5. Generación del PDF

La interfaz abre una vista previa y FacturaPdfService crea un documento A4 con número, socio, plan, vencimiento, estado y valor. jsPDF se importa únicamente cuando el usuario solicita el archivo, lo que evita cargar la biblioteca en el paquete inicial. El documento no contiene los requisitos tributarios completos y, en consecuencia, no debe presentarse como factura electrónica legal.

6. Configuración y seguridad

6.1. Archivo local de MongoDB

Para desarrollo se crea src/SistemaSuscripciones.Api/.env a partir de .env.example. Su estructura es la siguiente:

MONGODB__CONNECTIONSTRING=mongodb+srv://<usuario>:<contrasena>@<cluster>/?retryWrites=true&w=majority

MONGODB__DATABASENAME=SistemaSuscripcionesDb

MONGODB__COLLECTIONNAME=facturas

El archivo .env está excluido de Git. Las credenciales reales no deben aparecer en el repositorio, en documentación, capturas ni conversaciones. Si una contraseña de prueba fue expuesta, debe rotarse antes de compartir el proyecto. En producción, las variables deben configurarse en el administrador de secretos de la plataforma.

MongoDB Atlas exige un usuario de base de datos y una entrada válida en la lista de acceso de red. Los usuarios de Atlas y los usuarios de base de datos son identidades distintas (MongoDB, 2026a). Una regla 0.0.0.0/0 permite conexión desde cualquier origen y aumenta el

riesgo; debe sustituirse por direcciones confiables o conectividad privada cuando sea posible (MongoDB, 2026b).

6.2. Configuración SMTP

La pantalla Configuración permite guardar servidor, puerto, usuario, remitente, contraseña de aplicación, SSL/TLS y las dos plantillas. SmtpSettingsStore cifra el contenido con Data Protection y lo almacena en .smtp-settings.enc. Este mecanismo es apropiado para demostraciones locales, pero no constituye una estrategia productiva porque el sistema de archivos de un contenedor puede ser efímero y las claves de cifrado también requieren persistencia.

7. Despliegue local para revisión técnica

7.1. Prerrequisitos

• Git.

• SDK de .NET 10.

• Node.js 22.22.3 y npm 10 o versiones compatibles.

• Acceso a un clúster de MongoDB Atlas.

• Credenciales SMTP de prueba solo si se revisará el envío de correos.

7.2. Obtención e instalación

```
git clone https://github.com/SergioMesa2002/SistemaSuscripciones.git
cd SistemaSuscripciones
dotnet restore
cd src/SistemaSuscripciones.Web
nvm use
npm ci
cd ../../
```

npm ci utiliza package-lock.json y reproduce las versiones registradas. Si NVM todavía no tiene la versión indicada por .nvmrc, debe ejecutarse nvm install antes de nvm use.

7.3. Configuración de MongoDB

```
cp src/SistemaSuscripciones.Api/.env.example src/SistemaSuscripciones.Api/.env
```

A continuación se completa .env con un usuario de base de datos temporal, su contraseña y la cadena entregada por Atlas. La dirección IP del revisor debe agregarse a Network Access. La API debe reiniciarse después de cambiar el archivo.

7.4. Validación previa

```
dotnet build --maxcpucount:1
dotnet test tests/SistemaSuscripciones.Tests/SistemaSuscripciones.Tests.csproj --maxcpucount:1
cd src/SistemaSuscripciones.Web
npm run build
npm test -- --watch=false
cd ../../
```

En la última verificación del proyecto se aprobaron seis pruebas .NET y cuatro pruebas Angular. El archivo principal del backend es tests/SistemaSuscripciones.Tests/RecordatorioServiceTests.cs; el frontend se valida desde src/SistemaSuscripciones.Web/src/app/app.spec.ts.

7.5. Modalidades de ejecución

API únicamente. Útil para Swagger o Postman.

```
dotnet run --project src/SistemaSuscripciones.Api --launch-profile https
```

Swagger: https://localhost:7275/swagger

Frontend y backend separados. Requiere dos terminales y ofrece recompilación automática de Angular.

```
Terminal 1: dotnet run --project src/SistemaSuscripciones.Api --launch-profile https
Terminal 2: cd src/SistemaSuscripciones.Web && npm start
```

Interfaz: http://localhost:4200

Aplicación integrada. Compila Angular en wwwroot e inicia ASP.NET Core en una terminal.

```
./ejecutar-integrado.sh
```

Aplicación: https://localhost:7275

7.6. Secuencia de revisión funcional

1. Abrir la aplicación y confirmar que GET /api/Facturas responde sin error.

2. Crear una factura con estado PrimerRecordatorio.

3. Editar sus datos y confirmar que UltimoCorreoEnviado se conserva.

4. Abrir la vista previa y descargar el PDF.

5. En Development, guardar una cuenta SMTP de prueba y enviar el mensaje de comprobación.

6. Enviar el primer recordatorio y comprobar el cambio a SegundoRecordatorio.

7. Enviar el segundo recordatorio y comprobar el cambio a Desactivado.

8. Verificar que una factura desactivada no admita más envíos.

9. Eliminar una factura y confirmar la respuesta 204.

8. Ruta de despliegue productivo en Render

8.1. Estado actual y estrategia recomendada

El repositorio no contiene actualmente un Dockerfile ni un render.yaml. Por consiguiente, los pasos de esta sección describen la ruta recomendada y las condiciones de aceptación; no deben interpretarse como evidencia de que el despliegue productivo ya fue implementado.

Render no ofrece .NET entre sus runtimes nativos publicados y recomienda Docker cuando el lenguaje no está soportado de forma nativa o cuando se requieren compilaciones reproducibles (Render, 2026a). Para esta solución conviene un contenedor multietapa: una etapa Node compila Angular, una etapa del SDK publica la API y una etapa runtime ejecuta el resultado.

8.2. Artefactos que deben incorporarse

• Dockerfile multietapa en la raíz del repositorio.

• .dockerignore para excluir .git, node_modules, bin, obj, .env y archivos SMTP.

• Configuración del puerto para escuchar en 0.0.0.0 y usar la variable PORT.

• Una ruta de comprobación de salud que no dependa de Swagger.

• Configuración SMTP productiva basada en variables o un almacén de secretos.

• Persistencia explícita de claves Data Protection si se conserva información cifrada en disco.

8.3. Construcción esperada del contenedor

1. Usar una imagen Node compatible con Angular 22 para ejecutar npm ci y npm run build:integrated.

2. Usar mcr.microsoft.com/dotnet/sdk:10.0 para restaurar y publicar SistemaSuscripciones.Api en Release.

3. Usar mcr.microsoft.com/dotnet/aspnet:10.0 como imagen final.

4. Copiar la publicación, incluida wwwroot, a la imagen final.

5. Iniciar la API escuchando en http://0.0.0.0:$PORT.

8.4. Creación del servicio en Render

1. Publicar en Git únicamente código y archivos de construcción; no incluir .env ni secretos.

2. En Render, seleccionar New y luego Web Service.

3. Conectar el repositorio GitHub SergioMesa2002/SistemaSuscripciones.

4. Seleccionar Docker como lenguaje y confirmar la ruta del Dockerfile.

5. Elegir región y plan de cómputo conforme a la revisión.

6. Configurar las variables de entorno del servicio.

7. Definir una ruta de health check y crear el servicio.

8. Revisar los registros de compilación y confirmar que Render detecta el puerto público.

Render construye el Dockerfile conectado al repositorio y exige que el servicio web escuche en 0.0.0.0, preferiblemente en el puerto indicado por PORT (Render, 2026b). La terminación TLS pública la realiza el balanceador de Render.

8.5. Variables del servicio

ASPNETCORE_ENVIRONMENT: Production.

SERVEANGULAR: true.

MONGODB__CONNECTIONSTRING: Cadena secreta de Atlas; nunca debe versionarse.

MONGODB__DATABASENAME: SistemaSuscripcionesDb.

MONGODB__COLLECTIONNAME: facturas.

PORT: Puerto proporcionado por Render; el proceso debe escucharlo.

Render permite administrar variables desde la sección Environment y recomienda este mecanismo para proteger credenciales frente al código fuente (Render, 2026c). Los secretos no deben referenciarse como argumentos de construcción porque podrían incorporarse a la imagen.

8.6. MongoDB Atlas desde Render

Debe autorizarse la salida de red del servicio hacia Atlas. Durante una demostración podría utilizarse temporalmente una regla amplia, pero la revisión debe registrar el riesgo. Para producción se prefieren rangos de salida conocidos, peering o endpoints privados. MongoDB establece que las conexiones solo se aceptan desde entradas de la lista de acceso del proyecto (MongoDB, 2026b).

8.7. Restricción actual del correo en producción

ConfiguracionController devuelve 404 fuera de Development y SmtpSettingsStore inicia únicamente con valores locales predeterminados. En consecuencia, el despliegue productivo puede exponer el CRUD y la interfaz, pero el envío de recordatorios no debe considerarse habilitado hasta agregar un proveedor de configuración SMTP para variables o secretos. Montar solamente .smtp-settings.enc no es suficiente si las claves de Data Protection no persisten.

La decisión recomendada es conservar el flujo local actual para la demostración y crear una implementación productiva de SmtpSettings que lea secretos inyectados por la plataforma. Después deben añadirse pruebas de integración y validarse las restricciones de salida SMTP del proveedor elegido.

8.8. Verificación posterior al despliegue

1. Confirmar respuesta HTTPS de la raíz y carga de los archivos Angular.
2. Ejecutar GET /api/Facturas y revisar que la conexión a Atlas sea correcta.
3. Realizar creación, actualización y eliminación con datos no sensibles.

4. Confirmar que Swagger no se expone en Production con la configuración actual.

5. Revisar que los registros no impriman la cadena de conexión ni contraseñas.

6. Validar el health check y un reinicio del servicio.

7. Probar el correo únicamente después de implementar la configuración SMTP productiva.

9. Pruebas y criterios de aceptación

9.1. Backend

RecordatorioServiceTests.cs contiene seis pruebas xUnit. Estas verifican ambas transiciones, la prohibición de enviar cuando el ciclo terminó, la conservación del estado ante un fallo SMTP, el resultado para una factura inexistente y el filtrado del procesamiento masivo. Los dobles internos sustituyen al repositorio y al correo, de modo que la ejecución no necesita MongoDB ni SMTP.

```
dotnet test tests/SistemaSuscripciones.Tests/SistemaSuscripciones.Tests.csproj --maxcpucount:1
```

9.2. Frontend

app.spec.ts valida el comportamiento principal del componente Angular. Las pruebas se ejecutan con el builder unit-test, Vitest y jsdom.

```
cd src/SistemaSuscripciones.Web
npm test -- --watch=false
```

9.3. Criterios de aceptación de la revisión

• La solución .NET compila sin errores.

• Las seis pruebas del backend finalizan satisfactoriamente.

• Angular compila en modo producción y sus cuatro pruebas finalizan satisfactoriamente.

• El archivo .env y el almacenamiento SMTP no aparecen en Git.

• El CRUD responde con códigos HTTP coherentes.

• Las transiciones de recordatorio respetan el orden del negocio.

• La interfaz integrada funciona desde un único origen.

• Los errores públicos no exponen secretos ni trazas internas.

10. Riesgos, limitaciones y acciones pendientes

Hallazgo	**Impacto**	**Acción recomendada**
Credenciales de prueba expuestas previamente	Acceso no autorizado	Rotar usuario o contraseña antes de publicar.
Atlas configurado con 0.0.0.0/0	Superficie de ataque elevada	Restringir redes o usar conectividad privada.
SMTP productivo no externalizado	Recordatorios no operativos en Production	Crear configuración mediante secretos del proveedor.
Sin Dockerfile ni render.yaml	Despliegue no reproducible todavía	Incorporar y validar artefactos de contenedor.
Sin autenticación	Operaciones administrativas públicas	Agregar identidad, roles y autorización.
Procesamiento secuencial sin cola	Escalabilidad limitada	Agregar trabajos, reintentos e idempotencia si crece el volumen.

Hallazgo	Impacto	Acción recomendada
PDF administrativo	No cumple función tributaria	Añadir emisor, impuestos y requisitos legales antes de uso fiscal.

11. Conclusiones

La arquitectura separa adecuadamente el dominio, los casos de uso, los adaptadores externos, la API y la presentación. Las interfaces permiten probar la lógica de recordatorios de forma aislada, y el modo integrado ofrece una demostración sencilla sin eliminar el flujo de desarrollo con dos servidores. El tratamiento local de MongoDB y SMTP evita incorporar secretos al código, siempre que se respeten .gitignore y la rotación de credenciales expuestas.

Para la revisión técnica local, el proyecto dispone de instrucciones reproducibles, compilación, pruebas y tres modalidades de ejecución. Para producción, la opción coherente en Render es un contenedor multietapa, pero todavía deben implementarse el Dockerfile, el enlace al puerto dinámico, el health check y una fuente productiva de secretos SMTP. Estas actividades forman parte del cierre operativo y no alteran la arquitectura central propuesta.

Referencias

Angular. (2026). Version compatibility. https://angular.dev/reference/versions

Microsoft. (2026). Configuration in ASP.NET Core. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/configuration/?view=aspnetcore-10.0

MongoDB. (2026a). Connect to an Atlas cluster. MongoDB Documentation. https://www.mongodb.com/docs/atlas/connect-to-database-deployment/

MongoDB. (2026b). Configure IP access list entries. MongoDB Documentation. https://www.mongodb.com/docs/atlas/security/ip-access-list/

Render. (2026a). Docker on Render. Render Docs. https://render.com/docs/docker

Render. (2026b). Web services. Render Docs. https://render.com/docs/web-services

Render. (2026c). Environment variables and secrets. Render Docs. https://render.com/docs/configure-environment-variables